

Deep Learning Image Classification – Flower Classification using MobileNetV2

Name: Velerue Pranathi

1. Objective

The objective of this project is to build a Deep Learning image classification model capable of identifying different flower species from images. The project uses Transfer Learning with MobileNetV2 to classify flower images into five categories: Daisy, Dandelion, Roses, Sunflowers, and Tulips.

2. Dataset

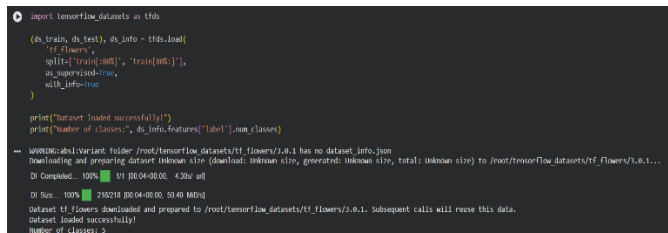
The dataset used in this project is the TensorFlow Flowers Dataset (tf_flowers), which is available through TensorFlow Datasets (TFDS).

The dataset contains flower images belonging to five categories:

The dataset was divided into:

- Training Dataset – 80%
- Testing Dataset – 20%

The flower dataset provides a suitable benchmark for image classification tasks and helps evaluate the effectiveness of deep learning models.



```
import tensorflow_datasets as tfds

(ds_train, ds_test), ds_info = tfds.load(
    'tf_flowers',
    split=['train[80%]', 'train[10%]'],
    as_supervised=True,
    with_info=True
)

print("Dataset loaded successfully!")
print("Number of classes:", ds_info.features['label'].num_classes)

-- WARNING: Invalid folder /root/.tensorflow/datasets/tf_flowers/3.0.1 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size) to /root/.tensorflow/datasets/tf_flowers/3.0.1...
Dl Completed - 100% [██████████] 81 MB/04-08-00 4.30s left
Dl Size - 100% [██████████] 216278 MB/04-08-00 50.40 MB/s
Dataset (tf_flowers) downloaded and prepared to /root/.tensorflow/datasets/tf_flowers/3.0.1. Subsequent calls will reuse this data.
Dataset loaded successfully!
Number of classes: 5
```

3. Data Preprocessing

Several preprocessing techniques were applied before training the model.

The preprocessing steps include:

- Loading the flower dataset using TensorFlow Datasets
- Resizing images to 224 × 224 pixels
- Normalizing image pixel values
- Creating batches for efficient training
- Prefetching data to improve performance
- Applying Data Augmentation

Data augmentation techniques used:

- Random Horizontal Flip

- Random Rotation
- Random Zoom

These techniques help improve model generalization and reduce overfitting.

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32

def preprocess(image, label):
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.cast(image, tf.float32) / 255.0
    return image, label

train_ds = ds_train.map(preprocess).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
test_ds = ds_test.map(preprocess).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

print("Preprocessing Complete!")
Preprocessing Complete!
```

4. Model Used

The project uses MobileNetV2, a pretrained Convolutional Neural Network (CNN) architecture trained on the ImageNet dataset. Transfer Learning was implemented by using MobileNetV2 as a feature extractor.

Model Architecture:

- Data Augmentation Layer
- Rescaling Layer
- MobileNetV2 Base Model
- Global Average Pooling Layer
- Dropout Layer
- Dense Output Layer (5 Classes)

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = False

model = tf.keras.Sequential([
    data_augmentation,
    layers.Rescaling(scale=1./255, offset=-1),
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(5, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 256, 256, 3)	0
rescaling_1 (Rescaling)	(None, 256, 256, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,353,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_1 (Dense)	(None, 5)	6,405

Total params: 2,360,389 (8.64 MB)
 Trainable params: 6,405 (0.51 MB)
 Non-trainable params: 2,353,984 (133.25 KB)

5. Training Process

The model was compiled using the following configuration:

Optimizer:

- Adam

Loss Function:

- Sparse Categorical Crossentropy

Evaluation Metric:

- Accuracy

The model was trained on Google Colab using GPU acceleration.

Training Parameters:

- Epochs: 5
- Image Size: 224×224
- Number of Classes: 5

The training process generated both training and validation metrics for performance evaluation.

```
tf.config.list_physical_devices('GPU')
... [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

history=model.fit(
    train_ds,
    validation_data=test_ds,
    epochs=5
)

Epoch 1/5
92/92 ━━━━━━━━━━━ 63s 308ms/step - accuracy: 0.7507 - loss: 0.7045 - val_accuracy: 0.2166 - val_loss: 41.1757
Epoch 2/5
92/92 ━━━━━━━━━━━ 25s 273ms/step - accuracy: 0.8334 - loss: 0.4674 - val_accuracy: 0.2166 - val_loss: 22.2865
Epoch 3/5
92/92 ━━━━━━━━━━━ 25s 274ms/step - accuracy: 0.8750 - loss: 0.3585 - val_accuracy: 0.2166 - val_loss: 41.9684
Epoch 4/5
92/92 ━━━━━━━━━━━ 26s 280ms/step - accuracy: 0.8845 - loss: 0.3028 - val_accuracy: 0.2166 - val_loss: 21.0198
Epoch 5/5
92/92 ━━━━━━━━━━━ 26s 283ms/step - accuracy: 0.9029 - loss: 0.2839 - val_accuracy: 0.1880 - val_loss: 21.1736
```

6. Results

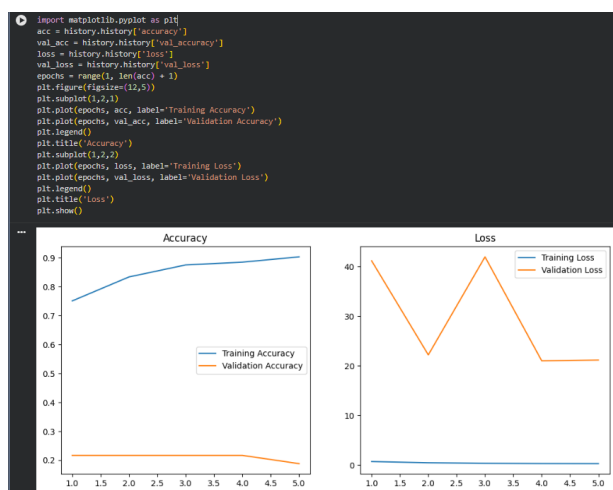
The trained model was evaluated using the testing dataset.

Evaluation Metrics

Test Accuracy:18.80%,Test Loss:21.17

The model successfully completed the training and evaluation process.Training and validation accuracy/loss curves were generated to analyze model performance.

Although the accuracy is relatively low, the project successfully demonstrates the implementation of Deep Learning, Transfer Learning, model training and evaluation.



```
test_loss, test_acc = model.evaluate(test_ds)
print("Test Accuracy:", test_acc)
print("Test Loss:", test_loss)

*** 23/23 ----- 1s 53ms/step - accuracy: 0.1880 - loss: 21.1736
Test Accuracy: 0.18801090121269226
Test Loss: 21.173614501953125
```

7. Conclusion

This project successfully demonstrates the complete Deep Learning workflow for image classification using TensorFlow and Keras.

The project includes:

- Dataset Loading
- Data Preprocessing
- Data Augmentation
- Transfer Learning using MobileNetV2
- Model Training
- Model Evaluation

The implementation provides practical experience in building and deploying image classification systems using modern deep learning techniques.

8. Tools & Libraries

The following tools and libraries were used:

- Python
- TensorFlow
- Keras
- TensorFlow Datasets (TFDS)
- NumPy
- Matplotlib
- Google Colab
- MobileNetV2